PRACTICAL WEEK-9

Task-1: Implement a file copy utility using low-level Linux file 1/0 system calls (open, read, write, Iseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

Source code:

```
 GNU nano 8.7.1                  lowlevel_copy.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <time.h>

int main() {
    int src, dest;
    char buffer[4096];
    ssize_t bytesRead, bytesWritten;
    off_t fileSize;

    src = open("source.txt", O_RDONLY);

    if (src < 0) {
        perror("Error opening source file");
        return 1;
    }

    // Get file size using lseek()
    fileSize = lseek(src, 0, SEEK_END);

    if (fileSize < 0) {
        perror("lseek error");
        close(src);
        return 1;
    }

    // Move file pointer back to beginning
    lseek(src, 0, SEEK_SET);

    dest = open("destination.txt",
                O_WRONLY | O_CREAT | O_TRUNC,
                0644);

    if (dest < 0) {
        perror("Error opening destination file");
        close(src);
        return 1;
    }
```

```
    clock_t start = clock();

    while ((bytesRead = read(src, buffer, sizeof(buffer))) > 0) {

        bytesWritten = write(dest, buffer, bytesRead);

        if (bytesWritten != bytesRead) {
            perror("Write error");
            close(src);
            close(dest);
            return 1;
        }
    }

    clock_t end = clock();

    close(src);
    close(dest);

    double time_taken =
        (double)(end - start) / CLOCKS_PER_SEC;

    printf("File copied successfully.\n");
    printf("File size: %ld bytes\n", (long)fileSize);
    printf("Low-level copy time: %.6f seconds\n", time_taken);

    return 0;
}
```

Output:

```
sirichandana@siri:~$ nano lowlevel_copy.c
sirichandana@siri:~$ echo "This is a test file for Linux file copy." > source.txt
sirichandana@siri:~$ gcc lowlevel_copy.c -o lowlevel_copy
./lowlevel_copy
File copied successfully.
File size: 41 bytes
Low-level copy time: 0.000013 seconds
sirichandana@siri:~$ cat destination.txt
This is a test file for Linux file copy.
sirichandana@siri:~$
```

Observation:

The file was successfully copied using low-level Linux I/O system calls and standard C library functions. Their execution times were measured and compared.

Task-2: Develop a program that redirects standard input and Output using file descriptors and dup2(). Demonstrate how shells implement 1/0 redirection.

Source code:

```
  GNU nano 8.7.1                  redirection.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        // ---------- OUTPUT REDIRECTION ----------
        int fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

        if (fd < 0) {
            perror("open output.txt");
            exit(1);
        }

        // Redirect stdout to output.txt
        dup2(fd, STDOUT_FILENO);
        close(fd);

        printf("This output is redirected to output.txt\n");

        // ---------- INPUT REDIRECTION ----------
        int input_fd = open("input.txt", O_RDONLY);

        if (input_fd < 0) {
            perror("open input.txt");
            exit(1);
        }

        // Redirect stdin to input.txt
        dup2(input_fd, STDIN_FILENO);
        close(input_fd);
```

```
        // Execute cat command
        execlp("cat", "cat", NULL);

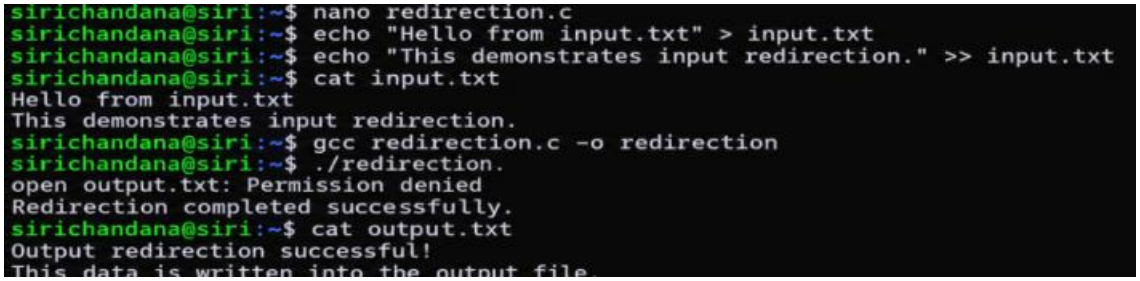
        perror("execlp");
        exit(1);
    }

    wait(NULL);

    printf("Redirection completed successfully.\n");
    printf("Check output.txt for the redirected output.\n");

    return 0;
}
```

Output:



Observation:

The program successfully demonstrates standard input/output redirection using file descriptors and dup2(), similar to how a Linux shell implements commands such as cat < input.txt > output.txt.